

Swift / ...

Protocol

Equatable

A type that can be compared for value equality.

iOS 8.0+ | iPadOS 8.0+ | Mac Catalyst 13.0+ | macOS 10.10+ | tvOS 9.0+ | visionOS 1.0+ |
watchOS 2.0+

protocol Equatable : ~Copyable, ~Escapable

Mentioned in

Adopting Common Protocols

Overview

Types that conform to the `Equatable` protocol can be compared for equality using the equal-to operator (`==`) or inequality using the not-equal-to operator (`!=`). Most basic types in the Swift standard library conform to `Equatable`.

Some sequence and collection operations can be used more simply when the elements conform to `Equatable`. For example, to check whether an array contains a particular value, you can pass the value itself to the `contains(_:)` method when the array’s element conforms to `Equatable` instead of providing a closure that determines equivalence. The following example shows how the `contains(_:)` method can be used with an array of strings.

```
let students = ["Kofi", "Abena", "Efua", "Kweku", "Akosua"]

let nameToCheck = "Kofi"
if students.contains(nameToCheck) {
    print("\(nameToCheck) is signed up!")
} else {
    print("No record of \(nameToCheck).")
}

// Prints "Kofi is signed up!"
```

Conforming to the Equatable Protocol

Adding `Equatable` conformance to your custom types means that you can use more convenient APIs when searching for particular instances in a collection. `Equatable` is also the base protocol for the `Hashable` and `Comparable` protocols, which allow more uses of your custom type, such as constructing sets or sorting the elements of a collection.

You can rely on automatic synthesis of the `Equatable` protocol’s requirements for a custom type when you declare `Equatable` conformance in the type’s original declaration and your type meets these criteria:

- For a `struct`, all its stored properties must conform to `Equatable`.